

# Thaisausage ERP Integration Operations Guide

**Document version:** 1.0.0  
**Date:** 12 September 2026  
**Audience:** Thaisausage IT / operations / deployment team  
**Domain:** https://thaisausage.krs.co.th

## 1. System purpose

ระบบนี้เป็น middleware ระหว่าง ERP ที่เก็บข้อมูลใน SQL Server กับ eVRP การออกแบบใช้ Schedule ดึงเฉพาะ SO ที่ Approve แล้วด้วย SELECT-only ก่อน map เป็น JSON และส่งไป eVRP เมื่อเปิด live mode หลังผ่านการอนุมัติและทดสอบ

ERP Webhook -> erp\_hooks inbox -> SQL Server SELECT -> mapper -> submission guard -> eVRP  
ERP D0 -> do\_receipts staging -----> operator review

## 2. Production deployment currently installed

| Component        | State                                             |
|------------------|---------------------------------------------------|
| VPS              | Linux host krs-cloud                              |
| Runtime          | Docker 29.8 / Compose 5.5                         |
| App              | Python container, non-root appuser , health check |
| Reverse proxy    | Caddy 2 with automatic HTTPS                      |
| Public URL       | https://thaisausage.krs.co.th                     |
| Persistent state | Docker volume for SQLite                          |
| Current mode     | dry_run=true , sync.enabled=false                 |

GET /health is public and reports process state only; it does not prove SQL/eVRP readiness. /docs and /openapi.json expose contracts but no credentials or payload history.

## 3. Configuration ownership

Application secrets live only in /opt/thaisausage/.env on the VPS with mode 600 . The file is not in GitHub. Keep it separate from deploy/vps.env , which contains VPS connection details.

Required application variables:

| Variable                                    | Purpose                                                                                  |
|---------------------------------------------|------------------------------------------------------------------------------------------|
| THAISAUSAGE_API_KEY                         | authenticates ERP to middleware                                                          |
| VRP_TOKEN                                   | authenticates middleware to eVRP as X-Token                                              |
| ERP_SQLSERVER_CONNECTION_STRING             | server/database/encryption portion of ODBC config                                        |
| ERP_SQL_USER / ERP_SQL_PASSWORD             | SQL Server read-only credential                                                          |
| ERP_SQLSERVER_WRITE_CONNECTION_STRING       | ODBC config for the DO write account (unused while <code>do_write.enabled=false</code> ) |
| ERP_SQL_WRITE_USER / ERP_SQL_WRITE_PASSWORD | least-privilege DO write credential; must differ from the read-only pair                 |

`sqlserver.approved_orders_query` is the only query used by scheduled SO sync. It must be a reviewed, parameter-free SELECT that filters only ERP-approved SOs.

SQL password is appended by the connector at runtime and is not stored in the base connection string. Use `Encrypt=yes;TrustServerCertificate=no` with a trusted certificate. Do not log the connection string.

`config/local.json` is a non-secret runtime file. It must define `source`, SQL settings, approved query set, mapping, `dry_run` and `sync.enabled`. Keep `dry_run=true` and `sync.enabled=false` until read-only SO verification is complete.

## 4. API responsibilities

| Endpoint                                          | Responsibility                | State effect                                                  |
|---------------------------------------------------|-------------------------------|---------------------------------------------------------------|
| POST <code>/api/v1/erp/hooks/order-ready</code>   | legacy compatibility receiver | stores <code>erp_hooks</code> ; not used by scheduled SO sync |
| POST <code>/api/v1/erp/orders</code>              | direct standard SO ingestion  | validates and submits through guard                           |
| POST <code>/api/v1/erp/pull</code>                | legacy/configured REST pull   | reads configured ERP REST endpoint                            |
| POST <code>/api/v1/erp/do-received</code>         | DO staging                    | adds <code>do_receipts</code> , never writes ERP              |
| GET <code>/api/v1/submissions/{request_id}</code> | delivery status               | read-only                                                     |
| GET <code>/health</code>                          | liveness                      | read-only                                                     |

The scheduled worker runs every `sync.interval_seconds` when `sync.enabled=true`. It executes only the reviewed `sqlserver.approved_orders_query`, which must filter SOs with `IsApprSo = 1`, maps the returned rows and uses the same submission guard. The old webhook is no longer the primary trigger. Each cycle is bounded by the approved query and already-claimed SOs are skipped.

## 5. SQL Server safety contract

---

The source credential is SELECT-only. The connector:

- lazily loads `pyodbc` and Microsoft ODBC Driver 18;
- requires a configured, reviewed query;
- permits only statements beginning with `SELECT` ;
- rejects write/side-effect keywords including INSERT, UPDATE, DELETE, MERGE and EXEC;
- binds order parameters instead of concatenating values;
- never accepts SQL text from an HTTP request;
- does not use `NOLOCK` or alter SQL Server settings.

During all tests, only SOs explicitly approved by the user may be selected from real ERP. All other data, delivery, retry, callback and error tests use mocks/staging. No ERP DDL or mutation is allowed.

## 6. Local state model

---

The current foundation creates:

```
submissions(request_id, payload_hash, state, result, created_at)
order_claims(order_no, request_id)
erp_hooks(event_id, event_type, source_id, company_id, order_no, changed_at, state,
received_at)
do_receipts(receipt_id, do_no, payload_hash, payload, state, received_at)
```

The current implementation protects request/event/receipt identity and stores remote VRP references when present. It does not yet provide a durable outbox with immutable payload bytes, attempt history, checkpoint leases or operator replay. Those are required before unattended production sync.

## 7. Operating procedures

---

### Check service

```
cd /opt/thaisausage
docker compose -f deploy/docker-compose.yml ps
docker compose -f deploy/docker-compose.yml logs --tail=100 thaisausage
curl -fsS https://thaisausage.krs.co.th/health
```

### Inspect Swagger

Open `https://thaisausage.krs.co.th/docs` and use the OpenAPI contract. Authenticated operations require the internal API key. Never paste a real key into a shared screenshot.

## Update an image

```
cd /opt/thaisausage
git fetch origin main
git reset --hard origin/main
docker compose -f deploy/docker-compose.yml build
docker compose -f deploy/docker-compose.yml up -d
docker compose -f deploy/docker-compose.yml ps
```

Before update: back up the SQLite volume, verify `.env` remains outside Git, and keep sync disabled.  
After update: check health, logs, OpenAPI and a mock webhook/DO request.

## Pause live behavior

Set `sync.enabled=false` and restart the app. If live sending is enabled, pause the process before restoring an old database backup. Reconcile accepted remote requests first; restoring old claims can cause duplicate attempts.

## 8. Monitoring and alerts

---

Minimum checks:

- HTTPS certificate and Domain availability
- app/Caddy container health
- SQL connection/query latency once enabled
- oldest queued/review hook age
- `needs_review`, `review`, `rejected` counts
- SQLite volume free space and backup freshness
- authentication failures and eVRP upstream errors

Do not log API keys, SQL passwords, full SO payloads, customer phone numbers or addresses. Use request/event IDs and redacted error codes for correlation.

## 9. DO staging and accounting boundary

---

`/api/v1/erp/do-received` is an observation/staging receiver. It never inserts, updates or deletes ERP data. The stored payload is for format discovery and reconciliation only. A future accounting export or ERP write requires a separate vendor contract, permission, idempotency key, reversal rule and approval.

COD callback from eVRP also requires an agreed ingress authentication method and duplicate identity. Do not enable a production callback route merely because the JSON shape is known.

## DO writer (`do\_write`) — CODE DONE, disabled

Since 16-09-26 the codebase contains a writer for `tbl_D0hdr / tbl_Dodt1` (`thaisausage/do_writer.py`). It is off by default and has never written to ERP. Field mapping is still incomplete; see [DO insert contract](do-insert-contract.md).

Operator rules:

- Keep `do_write.enabled=false` on the VPS. Deploy the code with the flag off; enabling is a separate approved change.
- The writer refuses to start if its credential environment variables reuse the read-only pair.
- No public endpoint exposes the writer. It is an internal call only.
- Writer states are recorded in `do_writes`: `preview`, `disabled`, `rejected`, `inserted`, `needs_review`.
- `inserted` and `needs_review` are never rewritten automatically. Investigate `needs_review` in ERP by

`TransactionNo / DoNo` before any manual action; a commit timeout may mean the rows already exist.

- Duplicate protection: `receipt_id` primary key plus unique `do_no` and `transaction_no`.

## 10. Release gates

---

### Gate A — read-only SQL discovery

- ☐ ODBC driver and pyodbc verified on runtime image
- ☐ SQL version, schema, source keys and timezone recorded
- ☐ Read-only service identity verified
- ☐ Approved SO sample scope recorded
- ☐ Query plan and row counts recorded without mutation

### Gate B — mapping and dry-run

- ☐ Header/detail/customer joins match ERP
- ☐ COD, credit, gift, Thai text, decimal and multiple-line cases match
- ☐ `dry_run=true` preview has no outbound eVRP request
- ☐ No source checkpoint advances during preview

### Gate C — controlled eVRP UAT

- ☐ eVRP test appointment and test token confirmed

- ☐ Mock tests and local recovery tests pass
- ☐ Controlled test records approved by business owner
- ☐ Remote identifiers and amounts reconcile with source
- ☐ User confirmation recorded

### Gate D-DO — ERP DO write (separate from eVRP release)

- ☐ DBA schema, constraints and mapping matrix signed
- ☐ Dedicated write credential issued and scoped to the two approved tables
- ☐ Approved test database or explicitly approved DO record available
- ☐ Mock transaction/rollback/duplicate tests pass
- ☐ Controlled UAT insert reconciled: Header count, Detail count and shared `TransactionNo`
- ☐ No unrelated ERP rows changed
- ☐ ERP owner and business/data owner sign-off recorded before `do_write.enabled=true`

### Gate D — live operation

- ☐ Backup/restore drill passed
- ☐ TLS/firewall/secret rotation documented
- ☐ Runbook and escalation owners assigned
- ☐ SQL sync enabled only for agreed scope
- ☐ First operating cycle observed before expansion

## 11. Recovery rules

---

`needs_review` means the outcome may be unknown. Check eVRP using its business reference before replay. Never delete `order_claims` to force a resend. A changed SO after send becomes an amendment review item; there is no documented update/cancel contract in the current integration package.

For a failed deployment, pause sender, preserve the database volume and inspect the last request/event IDs. Do not roll back by silently replacing the database or resetting checkpoints.

## 12. Document references

---

- [ERP Integration Guide](ERP\_Integration\_Guide.md)
- [OpenAPI JSON](openapi.json)

- [ERP source contract](erp-source-contract.md)
- [Deployment Compose](../deploy/docker-compose.yml)
- [SQL Server integration plan](../process/features/erp-sqlserver/active/ERP\_SQLSERVER\_PLAN\_12-09-26.md)

This document describes the current deployed foundation. It does not claim that SQL mapping, live eVRP delivery, accounting posting or automatic recovery has passed UAT.